

MNIST/CIFAR ML Classification

Lucas Giebler

4/19/2026

Section 1: MNIST Classification

The MNIST image dataset was loaded, with the first 60000 28x28 grayscale images split into the training set and the remaining 10000 samples into the testing set. All the data was normalized via min-max by dividing the pixel values by 255 (the maximum intensity value for each greyscale pixel). This normalization carried no risk of data leakage since we used an absolute maximum for pixels, which does not give the model any information about the testing data while training. Each sample was then labeled as lucky if it was a 3, 7, or 8, and labeled non-lucky otherwise. A 5-fold cross-validation was then conducted for 5 models for the task of binary classification of an image of a digit as lucky or non-lucky: stochastic gradient descent (SGD), logistic regression (LogReg), a single decision tree, random forest, and XGBoost. Accuracy, F1-score, ROC-AUC, and PR-AUC metrics were calculated for each model for each split.

The ensemble models, such as Random Forest and XGBoost, outperformed other linear models and the simple decision tree. Random Forest achieved the best overall performance, with an average accuracy of 0.9513, ROC-AUC of 0.9891, F1-score of 0.9164, and PR-AUC of 0.9781. XGBoost was second best, achieving an average accuracy of 0.9472, ROC-AUC of 0.9863, F1-score of 0.9105, and PR-AUC of 0.9728. The decision tree was the third best in performance, achieving an average accuracy of 0.9172, ROC-AUC of 0.9208, F1-score of 0.8617, and PR-AUC of 0.8385. Logistic Regression was fourth, achieving an average accuracy of 0.8747, ROC-AUC of 0.9315, F1-score of 0.7855, and PR-AUC of 0.8701. Finally, SGD was the worst-performing model, achieving an average accuracy of 0.8698, ROC-AUC of 0.9288, F1-score of 0.7882, and PR-AUC of 0.8656. The per-split metrics for each model were highly consistent with low variability, as indicated by the minor deviations of ± 0.01 across splits and the low standard deviations for each metric of every model. For example, Random Forest exhibits very low variability with a standard deviation of 0.0011 for ROC-AUC and 0.0021 for PR-AUC. The weaker models, such as SGD and Logistic Regression, also show small standard deviations from 0.004 to 0.009 across all metrics. The low variability across splits demonstrates that the Random Forest and XGBoost models consistently outperformed the others, and it was not due to the models getting lucky and performing well on just one split.

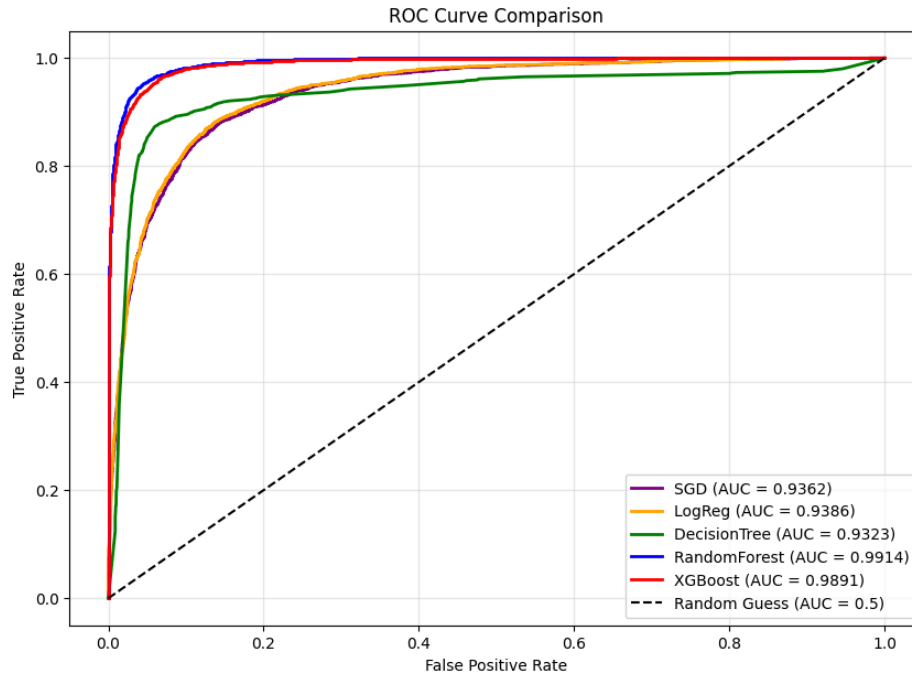


Figure 1. Plot of ROC curves for various models on the MNIST testing data.

After fitting the same models on the 60000 training samples, they were evaluated by generating the ROC and PR curves for each model. The shapes of the ROC curves in Figure 1 for every model indicate that each model performed far better than just random guessing, which would be the dotted diagonal line with a ROC-AUC of 0.5. Again, the results on the testing data match the 5-fold cross-validation results, with the Random Forest model performing the best with a ROC-AUC of 0.9914. The smooth, sharply rising curves of the ensemble models demonstrate that these models can consistently distinguish lucky digits from non-lucky ones, and are very close to a perfect model that would have a 100% true positive rate and 0% false positive rate (the point in the top left corner of the graph that would be a ROC-AUC of 1.0).

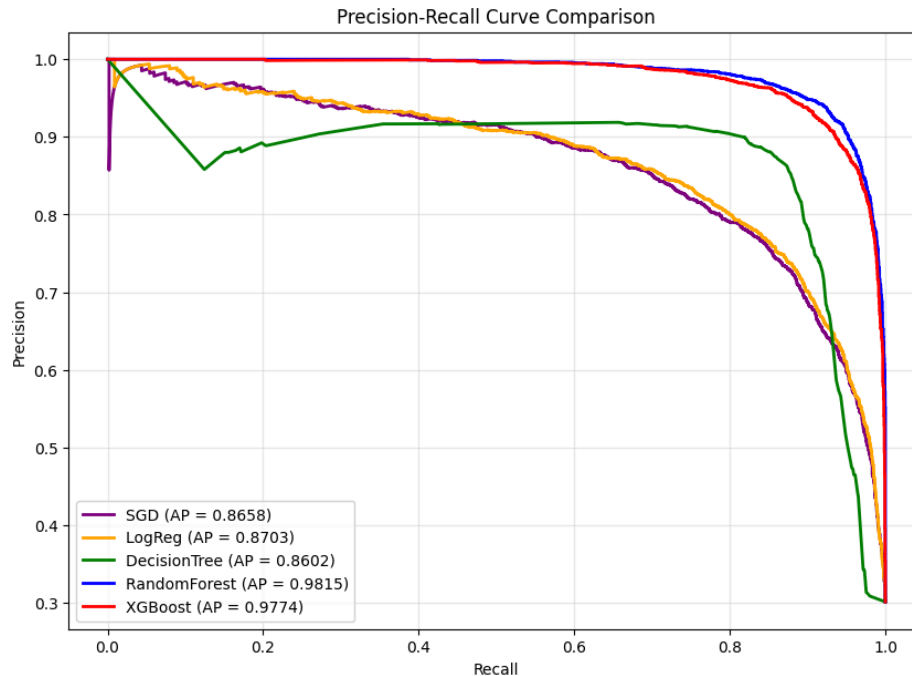


Figure 2. Plot of Precision-Recall curves for various models on the MNIST testing data.

Our dataset is imbalanced since we labeled only 3 out of 10 digits as lucky, so a Precision-Recall curve can reveal problems with our models stemming from imbalanced data, such as when the model simply predicts the majority class to attain high accuracy. When looking at the Precision-Recall curves in Figure 2, we similarly observe that the Random Forest model performs the best with a PR-AUC (AP) of 0.9815. Both the Random Forest and XGBoost PR curves demonstrate a fairly smooth graph where high precision corresponds with equally high recall, indicating that the models are not predicting the majority class despite the imbalance. The low precision despite high recall in the decision tree's Precision-Recall curve demonstrates that while the model may be correct in classifying lucky digits, it incorrectly predicts many non-lucky digits as lucky, indicating that the model is too liberal in classifying digits as lucky, rather than a problem with imbalanced data.

Overall, the Random Forest model performed the best, achieving the best balance between precision and recall and the highest AUC scores, indicating that it can consistently classify lucky versus non-lucky numbers and is a robust model. However, when breaking down performance by the individual lucky digits "3", "7", and "8", the models do not perform equally well across each class. Specifically, the "8" digit seems to be the most difficult to classify as lucky or non-lucky, consistently having the lowest metrics across every model, with the Random Forest model achieving 0.8541 ROC-AUC for "8" compared to 0.8807 for "3" and 0.9091 for "7", and 0.2493 PR-AUC for "8" compared to 0.2991 for "3" and 0.5291 for "7". This may suggest that specific features of the "8" digit may overlap with other digits like "6" or "9", making it harder for the classifiers to distinguish. Additionally, for each digit, every model's precision was very low, around 0.25 to 0.5, while their recall was high, around 0.8 to 0.9. This is due to how the models were trained to identify lucky digits "3", "7", and "8" combined whereas we now are trying to identify if a digit is a "3", "7", or "8" individually, so, for example, the models are classifying 7s

and 8s as positive in the “3” case, resulting in a high number of false positives and thus low precision.

Section 2: CIFAR-10 Classification

Next, a custom convolutional neural network based on ResNet was trained to classify images in the CIFAR-10 dataset as one of 10 classes: 'plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', or 'truck.' Each sample was a 32x32 RGB image. The dataset was split into training and testing sets, where the means and standard deviations were calculated for each RGB value in the training set. The individual RGB means and standard deviations were then used to normalize both the training and test sets. The training data was augmented using random cropping of size 32, random horizontal flips with a probability of 50%, and random affine transformations with rotations from -10 to 10 degrees and translations from -0.05 to 0.05 on both the x- and y-axes.

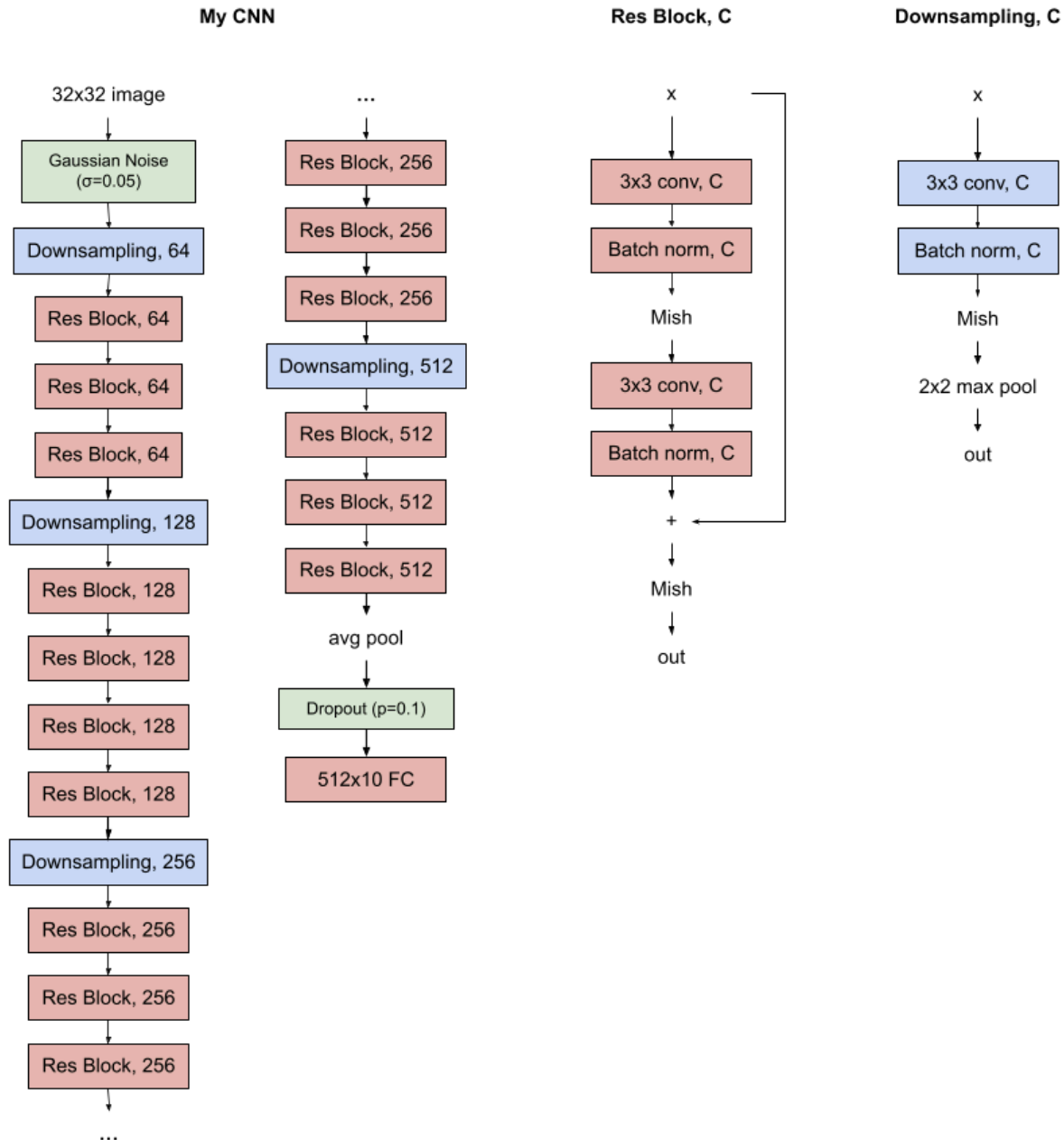


Figure 3. CNN architecture for the CIFAR10 classification problem. Left: High-level overview of the architecture of the CNN. Middle: Breakdown of each Residual Block in the CNN. Right: Breakdown of each Downsampling block in the CNN. C represents the number of channels.

The architecture for the CNN was based on that of ResNet and is diagrammed in Figure 3. Downsampling blocks were used between each stage to reduce the input size by a factor of 4 (2x2 max pool, dividing both width and height each by 2) each time, and every stage consisted of a sequence of residual blocks. Mish was used as the activation function to avoid the dying ReLU problem, where the sharp non-differentiable edge and the flat zero plateau could stop gradient updates, while Mish's smooth function with negative values allows negative gradients

to propagate to improve learning. At the very end of the architecture, before mapping to the 10 class scores with a linear layer, average pooling was conducted to average the value of each channel across the entire image to reduce the classification layer down to just 512x10 weights. The network was trained over 20 epochs with a learning rate of 0.01 and a batch size of 100. Given the use of data augmentation, added noise to the input, and dropout of parameters to counteract overfitting, a longer training time of 20 epochs was necessary to allow the model to fully plateau in test loss and converge to an optimum. The Adam optimizer was chosen to adaptively adjust learning rates per parameter and to use momentum to factor in gradients from previous iterations for faster convergence. A warm-up scheduler, OneCycleLR, was used with a maximum learning rate of 0.01 rather than cosine annealing for learning rate scheduling, since testing across various starting learning rates consistently resulted in trained models that achieved worse accuracy than the OneCycleLR scheduler on the test data (3-4% less accuracy) since it would either diverge (kept skipping over the minima) or never converge (took way too many epochs to train). The techniques that improved generalization accuracy on the testing data the most were Gaussian noise, dropout, and data augmentation. Additionally, the Gaussian noise perturbations and parameter dropouts on their own were not enough to prevent the model from overfitting by epoch 8 or 9, where the training loss would keep decreasing, but the test loss would begin increasing, resulting in a model that could only achieve around 87% test accuracy. After adding the data augmentation transformations mentioned above, the test loss consistently decreased over 20 epochs until reaching a final train loss of 0.1251 and a final test loss of 0.2682, and achieving a test accuracy of 91.7%.

Section 3: CIFAR-10 CNN Performance Evaluation

After loading the best model saved at the lowest test loss during training, the loaded network achieved the same test accuracy as at the end of training, of 91.7%.

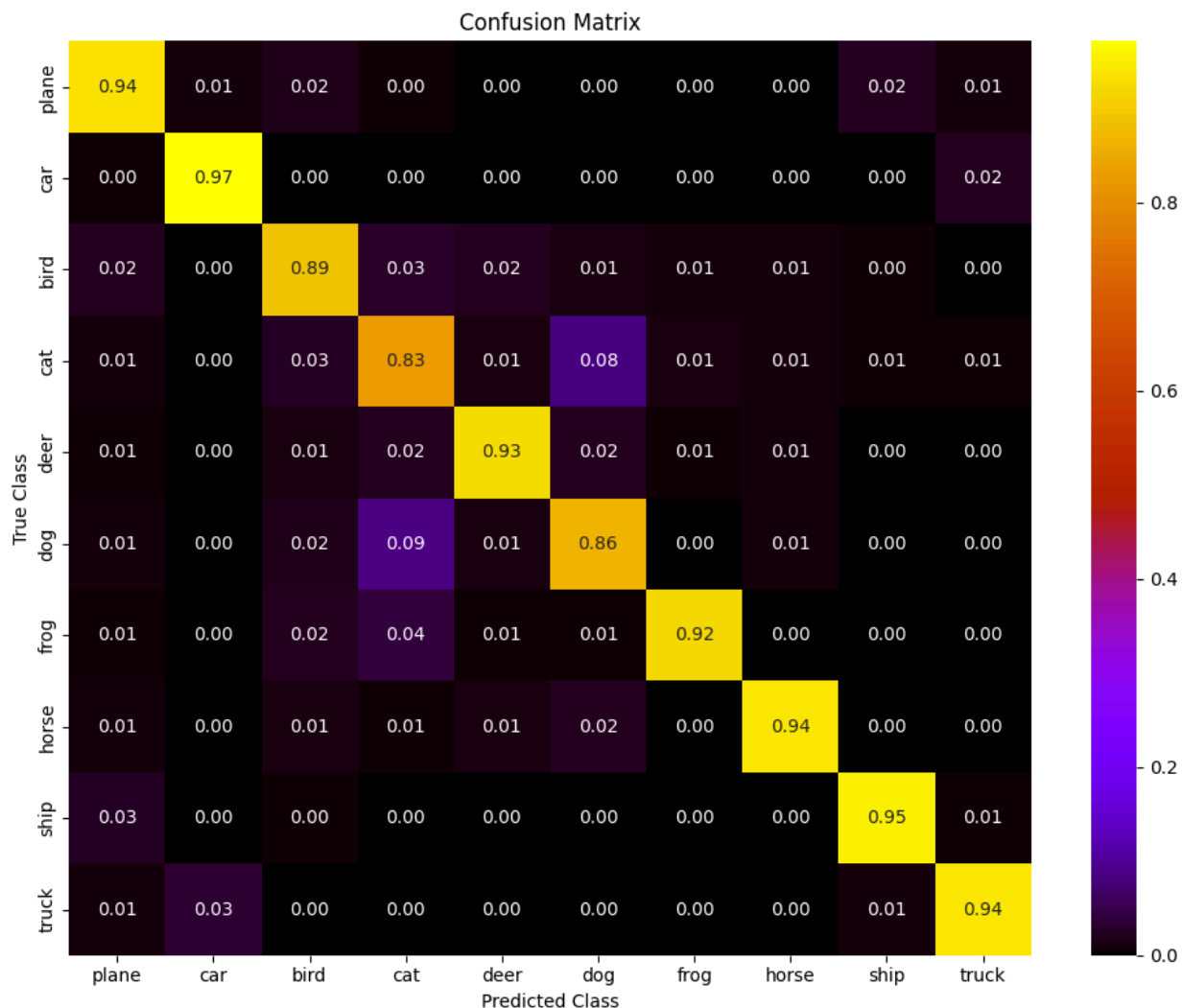


Figure 4. Confusion matrix in percentages of the loaded network's predicted versus actual class on the CIFAR-10 test dataset.

When looking at the confusion matrix in Figure 4, it seems that the network tends to frequently mistake the animals, particularly dogs and cats, for each other. The most common misclassification pairs were 'dog' predicted as 'cat' (9% of all true 'dog'), 'cat' predicted as 'dog' (8% of all true 'cat'), and many other misclassifications between several different animals, each constituting around ~3% of all predictions of that animal class. I suspect the ears of the cat likely contribute to the network's distinguishing cat features, as it's a very prominent feature for cats that many dogs could also share depending on the breed, and the fact that the network misclassified 'bird' as 'cat' for 3% of all true 'bird' indicates that the network may have also mistaken a bird's outstretched wings for cat ears. All these pairs make good sense, since many of the animals are very similar in features, having legs, ears, neck, eyes, etc, and many different configurations/contortions of their bodies could resemble different animals. The most misclassified pairs of animals are between cats and dogs, which is also unsurprising, since both can often be of roughly the same size, face structure, tail shape, and ear shape, which can make it difficult to find distinguishing features, especially in a low-resolution 32x32 image. Some

other non-animal common misclassifications were 'ship' predicted as 'plane' (3% of all 'plane' predictions), and 'truck' predicted as 'car' (3% of all 'car' predictions).

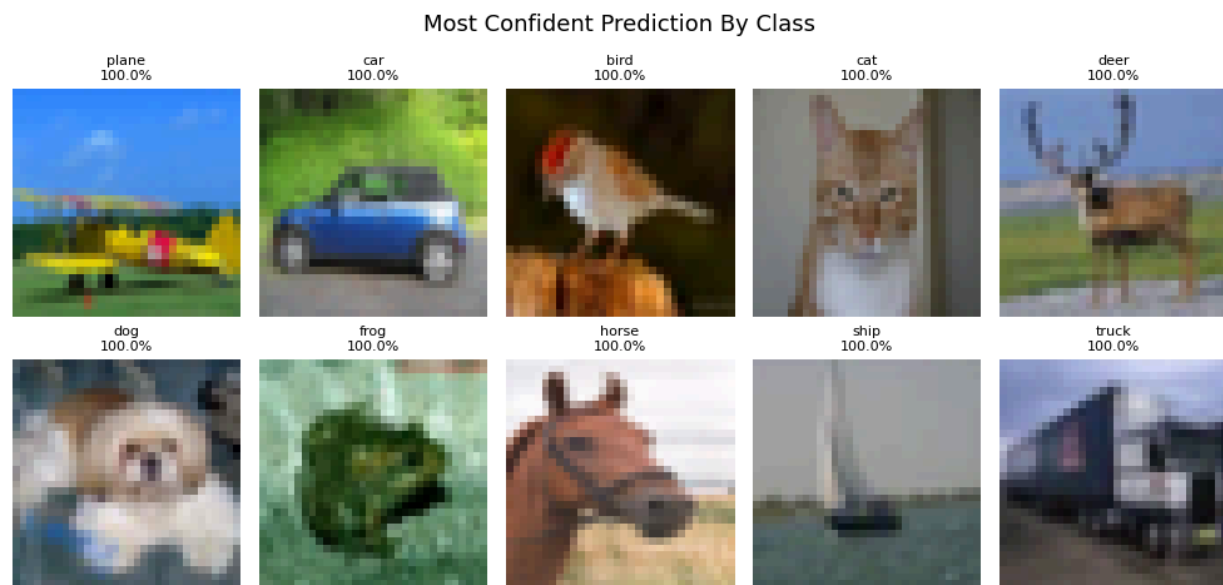


Figure 5. Sample images of the highest confidence predictions for each class by the loaded network.

When looking at the examples of images from each class in Figure 5, we can observe that the highest confidence cat example has distinct, pointy ears. The model is likely using the ears of the cat as a major distinguishing feature, which can very easily confuse the network with other animals that may have pointy features around their head (like the antlers of deer or just a pointy-eared dog breed). As for the dogs, the model probably distinguishes them using the facial features (large, prominent nose and mouth shape). For the plane, the most distinct features are likely the long, straight lines of its geometry and the tailfin of the plane. I can see now how 2% of all plane predictions were actually birds, since maybe some birds with long necks (like geese) and tail feathers could, in some cases, resemble a plane. The car's distinguishing features would likely be the wheels and its smaller, more aerodynamic shape/form. The truck's distinguishing features would be the boxy structure, bulkier frame, and long length. The distinct features of the deer could be its relatively thin and long four legs, along with its antlers. The horse's distinct features could be the harness or reins on the horse, as well as the mane of hair on the back of its head and neck. The bird's distinct features could be the two legs and the beak, or the prominent tail feathers sticking straight out. The frog's distinct features are a little harder since it seems to be just a blobby mess with eyes, which could also explain the slightly high misclassification of 'frog' for 'cat', since cats can often look similarly blobby, especially if the ears are missing from the frame or are hard to see in a particular image. However, the frog's distinct round eyes and the overall bulbous, triangular shape of the frog from the legs and head may be what the network uses to distinguish frogs. The sail or random geometry on top of the ship, combined with the smooth, curved bottom of the hull, is likely the most prominent feature to distinguish them.

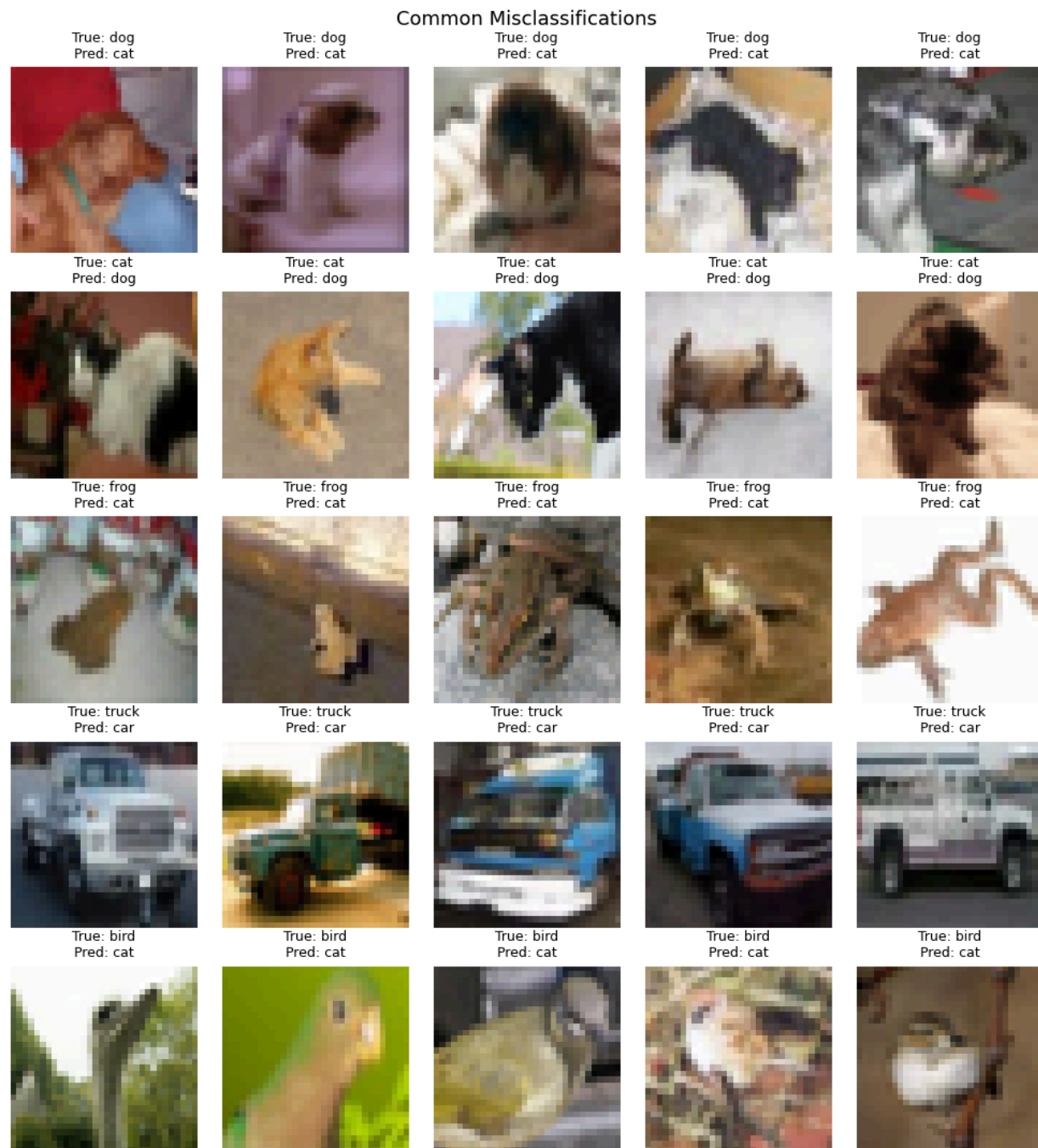


Figure 6. Examples for each of the 5 most common class misclassifications made by the loaded network.

The expectation that trucks were classified based on their boxiness and length is reinforced by looking at the examples of trucks misclassified as cars in Figure 6. All the examples of misclassified trucks don't have trailers or have them cut off, and are much smaller variants, designed more for individual/personal use rather than the typical large commercial trucks like semitrucks. In all the images of the cats misclassified as dogs, they all have their head angled in such a way, or they are far enough away from the camera that the shape of their

ears is a little difficult to make out, lending some explanation as to why the network classified them as dogs. As for the images of dogs predicted as cats, the last three images are extremely blurry and hard to make out, and the first two images have the dogs not showing their faces. Since the network was likely using facial features as determining features for dogs, it couldn't identify these cases as dogs since their faces were not clearly visible, and defaulted to predicting them as cats. All the images of birds predicted as cats are taken at strange close-ups or framed in a way that their legs are not visible, and the texture of the images looks somewhat like a furry mass, similar to the furry blobs that cats usually look like in training. The third and fourth images of frogs that were misclassified as cats were likely predicted as such since they had geometry that formed triangular shapes near their heads (the front legs for the 3rd image and eyes for the 4th image), similar to cat ears. The first two images of frogs are small and hard to make out, possibly confusing the network. Finally, the last frog image is likely a very rare instance of a frog extending its limbs that the network likely didn't have lots of training on, so it confused it for maybe a cat lying down with its legs and tail spread out. It seems that the network decides when an animal looks blob-like without any of the distinguishing features for any of the other animals, it defaults to classifying it as a cat, since maybe the cats it was trained on are always lying down or sitting in a manner that, when combined with the low 32x32 resolution, makes them seem just like a furry mass/blob.

Section 5: CNN Performance Tuning

Initially, a 3-stage (channels of 32->64->128) architecture was used with just a single linear layer at the end, but it would remain consistently around 75% accuracy when adjusting other hyperparameters. To achieve a higher accuracy percentage, I extended the depth of the CNN architecture from 3 stages to 4 and designed it along similar lines to the ResNet architecture.

Upon changing the structure to a 4-stage system (channels of 64->128->256->512), adding an average pooling step before the final linear layer, and adjusting the number of blocks per stage to 3->4->6->3, the test accuracy increased to ~91%. Each stage was separated by a downsampling block that passed the input through a 3x3 convolution, batch norm, and finally a 2x2 max-pooling to half the input's width and height, both by 2 (shrinking the input size by 4). Downsampling was needed to reduce computational cost and prevent overfitting by reducing the number of dimensions. Additionally, downsampling allowed the network to extract increasingly abstract features from the raw data, learning higher-level, more complex features rather than just low-level details (like edges).

Each residual block was the same structure as a standard residual block, holding out the original value of the input and adding it back after two convolutional passes just before the second activation. The only difference from a typical residual block structure was that ReLU was replaced with Mish to have a continuous activation function for nonlinearity to allow negative gradients to propagate when learning, instead of only zeros that wouldn't contribute anything. Residual blocks ensured that the network did not suffer from vanishing gradients, where values at the very beginning of the network were being forgotten later, deep in the network, resulting in extremely poor performance, around 50% accuracy on the testing data.

Each stage's downsampling block was followed by several residual blocks, where I determined the number of residual blocks at each stage to be the same as ResNet: 3->4->6->3 (two convolutions per residual block, so the number of convolutions would be: 6->8->12->6). More blocks were allocated in the middle of the architecture since the feature maps at that point are much smaller (less compute needed) and contain richer representations (can extract more meaningful information from more abstract/higher-level features). The channels for each stage doubled (64->128->256->512) to compensate for the lost spatial dimensionality from downsampling, basically trading spatial resolution for feature depth.

Overfitting was a major concern with a deep convolutional neural network architecture with millions of parameters. This was combated by adding Gaussian noise to the input of every image with a standard deviation of 0.05, applying dropout before the 512x10 classification layer with a probability of 0.1, and augmenting the training data via random cropping, horizontal flipping, random rotation, and random translation of the images. These measures resulted in the test loss staying with the train loss after each epoch during training and would consistently decrease over the course of 20 epochs. This indicated that the model was generalizing well by performing equally well on the test data when trained on the separate training set. Additionally, the warmup learning rate scheduler helped to speed up convergence, since choosing a learning rate around 0.01 would cause the network to be highly unstable and diverge, while a smaller learning rate around 0.001 would cause the network to take a very long time (>20 epochs) to train and only achieve around 88% train accuracy and 86% test accuracy. Including the scheduler allowed the network to gradually converge to an optimum over 20 epochs without diverging and not taking too long.